

## **AWS CLOUD LAB SERIES**

# **End-to-End Technical Documentation**

Lab 1 — AWS WAF

Lab 2 — AWS Systems Manager Parameter Store

Lab 3 — EC2 + Parameter Store Integration

**Region: us-east-1 (US East — N. Virginia)**

**Implementation status: COMPLETED**

**Environment: AWS Console + Amazon Linux 2023 EC2**

## 1. Document Overview

This document records the AWS lab work actually implemented and validated across three connected labs. Lab 1 establishes the web application and AWS WAF protection. Lab 2 introduces AWS Systems Manager Parameter Store, including String, StringList, SecureString, hierarchy, versions, labels, IAM permissions, and KMS configuration. Lab 3 integrates the existing EC2 instance with Parameter Store through an IAM role.

The documentation preserves the resource names, configuration values, commands, validation results, and troubleshooting observed during implementation. Sensitive credentials, secrets, and the user's public test IP are intentionally not reproduced.

## 2. Overall Architecture

Internet → Application Load Balancer → AWS WAF Web ACL → Target Group → Existing EC2/Apache

Existing EC2 → IAM Instance Role → AWS Systems Manager Parameter Store (/lab2/app/\*)

| Layer          | Resource                     | Purpose                                           |
|----------------|------------------------------|---------------------------------------------------|
| Networking     | waf-lab-vpc                  | Dedicated lab VPC                                 |
| Load balancing | waf-lab-alb                  | Internet-facing application entry point           |
| Security       | waf-lab-web-acl              | Regional AWS WAF protection for the ALB           |
| Compute        | waf-lab-web-server           | Amazon Linux 2023 Apache web server               |
| Configuration  | /lab2/app/*                  | Application configuration and demo secret storage |
| Identity       | Lab3-EC2-ParameterStore-Role | Allows EC2 to read Parameter Store                |

## 3. Lab 1 — AWS WAF Basic Lab

Status: COMPLETED

### 3.1 Objective

Implement a basic web application security architecture using an Application Load Balancer, Amazon EC2, and a Regional AWS WAF Web ACL. The lab demonstrated IP-based blocking, Count mode, managed protections, rule evaluation, monitoring, and HTTP 403 validation.

### 3.2 Infrastructure

| Component          | Configuration                                   |
|--------------------|-------------------------------------------------|
| VPC                | waf-lab-vpc / 10.0.0.0/16                       |
| Subnet 1           | waf-public-subnet-1 / 10.0.1.0/24 / us-east-1a  |
| Subnet 2           | waf-public-subnet-2 / 10.0.2.0/24 / us-east-1b  |
| Internet Gateway   | waf-lab-igw                                     |
| Route table        | waf-public-rt / 0.0.0.0/0 → Internet Gateway    |
| EC2                | waf-lab-web-server / Amazon Linux 2023          |
| EC2 security group | waf-lab-ec2-sg                                  |
| Target group       | waf-lab-tg / HTTP :80 / health check /          |
| ALB                | waf-lab-alb / Internet-facing / IPv4 / HTTP :80 |
| ALB security group | waf-lab-alb-sg                                  |
| WAF Web ACL        | waf-lab-web-acl / Regional / default Allow      |

### 3.3 WAF Configuration and Testing

The final Web ACL contained 14 configured rules, including AWS managed rule groups, custom rules, IP rules, rate-based/count-mode rules, and the lab IP blocking rules. The existing AWS Managed Core Rule Set was verified rather than duplicated.

| Item           | Observed configuration                                            |
|----------------|-------------------------------------------------------------------|
| Core rule set  | AWS-AWSManagedRulesCommonRuleSet; Default / Version 1.23; 700 WCU |
| First IP set   | waf-lab-test-ip-set                                               |
| First IP rule  | waf-lab-block-test-ip — Count after test                          |
| Second IP set  | waf-lab-second-ip-set — restored to 192.0.2.1/32                  |
| Second IP rule | waf-lab-second-rule — Block                                       |
| Blocking test  | Temporary test IP produced HTTP 403 Forbidden                     |
| Restoration    | Removing the temporary test IP restored normal access             |

### 3.4 Validation

| Test                    | Result                                  |
|-------------------------|-----------------------------------------|
| Application through ALB | PASS                                    |
| EC2 target health       | PASS — Healthy                          |
| WAF association         | PASS                                    |
| IP blocking             | PASS — HTTP 403 observed                |
| Restore normal access   | PASS                                    |
| WAF dashboard           | PASS — allowed/blocked traffic recorded |

Dashboard values observed for the displayed last-three-hour period were 8 total requests, 5 allowed, 3 blocked, 0 CAPTCHA, 0 Challenge, and 0 Monetized.

## 4. Lab 2 — AWS Systems Manager Parameter Store

Status: COMPLETED

### 4.1 Objective

Create and validate Parameter Store values using String, StringList, and SecureString types; demonstrate hierarchical naming, parameter versioning, labels, IAM permissions, and the AWS-managed SSM KMS key.

### 4.2 Parameters Created

| Parameter               | Type         | Tier     | Configuration                                                             |
|-------------------------|--------------|----------|---------------------------------------------------------------------------|
| /lab2/app/name          | String       | Standard | Initial value AWS-ParameterStore-Lab; edited to AWS-ParameterStore-Lab-v2 |
| /lab2/app/allowed-ports | StringList   | Standard | 80,443,8080                                                               |
| /lab2/app/db-password   | SecureString | Standard | Demo secret; value intentionally not reproduced                           |

### 4.3 Versioning and Labels

The /lab2/app/name parameter was edited, producing Version 2. Parameter history showed Versions 1 and 2. Version 2 was labeled 'current'.

### 4.4 KMS Configuration

The SecureString parameter used the current AWS account as the KMS source and the AWS-managed Systems Manager key identified as alias/aws/ssm.

### 4.5 IAM Read Policy

Policy name: Lab2-ParameterStore-ReadOnly

Policy ARN: arn:aws:iam::567487919764:policy/Lab2-ParameterStore-ReadOnly

The policy was attached directly to the IAM user used for CLI validation and allowed read-only Parameter Store operations for the lab hierarchy.

Final policy scope included both the hierarchy parent and its children so GetParametersByPath could be evaluated correctly:

- ssm:GetParameter
- ssm:GetParameters
- ssm:GetParametersByPath
- arn:aws:ssm:us-east-1:567487919764:parameter/lab2/app
- arn:aws:ssm:us-east-1:567487919764:parameter/lab2/app/\*

### 4.6 CLI Validation

CLI retrieval was successfully validated from Git Bash. Because Git Bash converted the leading parameter path into a Windows filesystem path, MSYS\_NO\_PATHCONV=1 was required for local commands such as:

```
MSYS_NO_PATHCONV=1 aws ssm get-parameter --name "/lab2/app/name" --region us-east-1
```

The String, StringList, SecureString (with decryption), and GetParametersByPath operations were validated. The SecureString value itself is not reproduced here.

## 5. Lab 3 — EC2 + Parameter Store Integration

Status: COMPLETED

### 5.1 Objective

Reuse the existing Lab 1 EC2 instance and allow it to retrieve Parameter Store values using an EC2 IAM role, without placing long-lived AWS access keys on the server.

### 5.2 IAM Role

| Setting           | Value                        |
|-------------------|------------------------------|
| Role              | Lab3-EC2-ParameterStore-Role |
| Trusted service   | EC2                          |
| Permission policy | Lab2-ParameterStore-ReadOnly |
| Attached instance | waf-lab-web-server           |

### 5.3 Validation from EC2

Inside the existing Amazon Linux EC2 terminal, `aws sts get-caller-identity` returned an assumed-role identity for Lab3-EC2-ParameterStore-Role, confirming that the instance was using the IAM role.

The following access tests succeeded:

```
aws ssm get-parameter --name "/lab2/app/name" --region us-east-1
```

```
aws ssm get-parameter --name "/lab2/app/allowed-ports" --region us-east-1
```

```
aws ssm get-parameter --name "/lab2/app/db-password" --with-decryption --region us-east-1 --query
"Parameter.Type"
```

```
aws ssm get-parameters-by-path --path "/lab2/app" --region us-east-1 --query
"Parameters[0].{Name:Name,Type:Type}"
```

The SecureString test was intentionally queried for its type so the secret value was not exposed.

## 6. Troubleshooting Records

| Issue                                     | Cause                                                                       | Resolution                                                                     |
|-------------------------------------------|-----------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Local SSM AccessDenied for /lab2/app/name | Git Bash MSYS path conversion changed the parameter name to a Windows path. | Used MSYS_NO_PATHCONV=1 for AWS CLI commands.                                  |
| GetParametersByPath AccessDenied from EC2 | Policy covered parameter children but not the parent path resource.         | Added both parameter/lab2/app and parameter/lab2/app/* to the IAM policy.      |
| WAF HTTP 403 during test                  | Temporary test IP matched the WAF Block rule.                               | Removed/restored the temporary IP-set test value; application access returned. |

## 7. Final End-to-End Validation

| Lab / Control                        | Status    |
|--------------------------------------|-----------|
| Lab 1 — ALB → EC2 application        | COMPLETED |
| Lab 1 — WAF Web ACL association      | COMPLETED |
| Lab 1 — IP blocking / 403 validation | COMPLETED |
| Lab 2 — String parameter             | COMPLETED |
| Lab 2 — StringList parameter         | COMPLETED |
| Lab 2 — SecureString parameter       | COMPLETED |
| Lab 2 — Version 2 + current label    | COMPLETED |
| Lab 2 — IAM read permissions         | COMPLETED |
| Lab 2 — CLI validation               | COMPLETED |
| Lab 3 — EC2 IAM role                 | COMPLETED |
| Lab 3 — EC2 Parameter Store access   | COMPLETED |
| Lab 3 — GetParametersByPath          | COMPLETED |

## 8. Security and Operational Notes

- The existing Lab 1 EC2 instance was reused; no second EC2 instance was created for Lab 3.

#### SUNITH – AWS SELF-LEARNING AND HANDS-ON IMPLEMENTATION ACTIVITY

- The ALB, target group, security groups, and WAF configuration were not changed as part of Lab 3.
- Parameter Store SecureString was used for the lab secret, and its plaintext value is intentionally omitted from this document.
- The EC2 retrieves Parameter Store values through an IAM instance role rather than storing long-lived AWS access keys on the server.
- The user's public test IP used during WAF validation is intentionally omitted.

## 9. Conclusion

The three-lab sequence was successfully completed. The resulting environment demonstrates a connected AWS workflow: a web application hosted on EC2 is exposed through an Application Load Balancer and protected by AWS WAF; application configuration and a demo secret are stored in Systems Manager Parameter Store; and the same EC2 instance accesses those values securely through an IAM instance role.

# Lab 1 — AWS WAF Basic Lab

## Document Overview

---

This document records the completed AWS WAF lab using an Application Load Balancer, Amazon EC2, and a Regional AWS WAF Web ACL. It preserves the actual resource names, configuration, testing results, and troubleshooting observed during implementation.

1. Objective
2. Purpose
3. Architecture
4. Prerequisites
5. Implementation
6. Configuration Details
7. Testing & Validation
8. Observations
9. Troubleshooting
10. Final Result
11. Conclusion
12. Completion Checklist

### 1. Objective

Implement a basic web application security architecture using an Application Load Balancer, Amazon EC2, and a Regional AWS WAF Web ACL. The lab demonstrated IP-based blocking, Count mode, managed protections, rule evaluation, monitoring, and HTTP 403 validation.

### 2. Purpose

- Configure an ALB and EC2 web application protected by AWS WAF.
- Demonstrate IP sets and IP-based access control.
- Validate Allow, Block, and Count behavior.
- Verify AWS Managed Rules and WAF monitoring.
- Perform a controlled 403 blocking test and restore normal access.

### 3. Architecture

Internet → Application Load Balancer → AWS WAF Web ACL → Target Group → EC2 / Apache

| Component        | Implemented Configuration                      |
|------------------|------------------------------------------------|
| VPC              | waf-lab-vpc / 10.0.0.0/16                      |
| Subnet 1         | waf-public-subnet-1 / 10.0.1.0/24 / us-east-1a |
| Subnet 2         | waf-public-subnet-2 / 10.0.2.0/24 / us-east-1b |
| Internet Gateway | waf-lab-igw                                    |
| Route Table      | waf-public-rt / 0.0.0.0/0 → IGW                |
| EC2              | waf-lab-web-server / Amazon Linux 2023         |
| Target Group     | waf-lab-tg / HTTP :80 / health check /         |
| ALB              | waf-lab-alb / Internet-facing / IPv4           |
| WAF              | waf-lab-web-acl / Regional / default Allow     |

## 4. Prerequisites

- AWS account and console access
- AWS Region us-east-1
- Basic familiarity with VPC, EC2, Security Groups, ALB, and AWS WAF
- Permission to create and configure required AWS resources

## 5. Implementation

### 5.1 Create the VPC

Created waf-lab-vpc with IPv4 CIDR 10.0.0.0/16.

### 5.2 Create Public Subnets

Created waf-public-subnet-1 (10.0.1.0/24, us-east-1a) and waf-public-subnet-2 (10.0.2.0/24, us-east-1b).

### 5.3 Internet Gateway and Route Table

Created waf-lab-igw and waf-public-rt with 0.0.0.0/0 routed to the Internet Gateway.

### 5.4 Launch and Configure EC2

Created waf-lab-web-server using Amazon Linux 2023, public connectivity, and waf-lab-ec2-sg. HTTP/80 was allowed from Anywhere IPv4 and SSH/22 from My IP. Apache and the custom lab page were configured.

### 5.5 Create Target Group and ALB

Created waf-lab-tg for HTTP port 80 with health check /. Created waf-lab-alb as an internet-facing IPv4 ALB using the two public subnets and forwarding HTTP traffic to the target group.

### 5.6 Create and Associate WAF

Created waf-lab-web-acl as a Regional Web ACL and associated it with waf-lab-alb.

### 5.7 Configure IP Blocking Test

Created waf-lab-test-ip-set and waf-lab-block-test-ip. The temporary test produced HTTP 403, after which the rule was changed to Count. A second IP set/rule pair was also tested; the final second test address was restored to 192.0.2.1/32.

## 6. Configuration Details

| Item                   | Observed Configuration                           |
|------------------------|--------------------------------------------------|
| Final WAF rule count   | 14                                               |
| Managed Core Rule Set  | AWS-AWSManagedRulesCommonRuleSet                 |
| Core Rule Set version  | Default / Version 1.23                           |
| Core Rule Set capacity | 700 WCU                                          |
| First IP set           | waf-lab-test-ip-set                              |
| First IP rule          | waf-lab-block-test-ip — Count after test         |
| Second IP set          | waf-lab-second-ip-set — restored to 192.0.2.1/32 |
| Second IP rule         | waf-lab-second-rule — Block                      |
| Default Web ACL action | Allow                                            |

## 7. Testing & Validation

| Test                     | Validation                                            | Result |
|--------------------------|-------------------------------------------------------|--------|
| Application availability | ALB displayed custom AWS WAF Lab page                 | PASS   |
| Target health            | EC2 target verified Healthy                           | PASS   |
| WAF association          | waf-lab-web-acl associated with waf-lab-alb           | PASS   |
| IP blocking              | Temporary IP-set test produced HTTP 403               | PASS   |
| Restore access           | Restoring IP-set configuration returned normal access | PASS   |
| Dashboard validation     | Allowed/blocked traffic recorded                      | PASS   |

Dashboard values observed for the displayed last-three-hour period: 8 total requests, 5 allowed, 3 blocked, 0 CAPTCHA, 0 Challenge, and 0 Monetized.

## 8. Observations

- The ALB successfully routed traffic to the EC2 web server.
- The EC2 target remained healthy.
- A Regional WAF Web ACL protected the ALB directly without CloudFront.
- A matching Block rule produced HTTP 403 Forbidden.
- Count mode allowed traffic observation without active blocking.
- AWS Managed Rules provided broader application-layer protections.

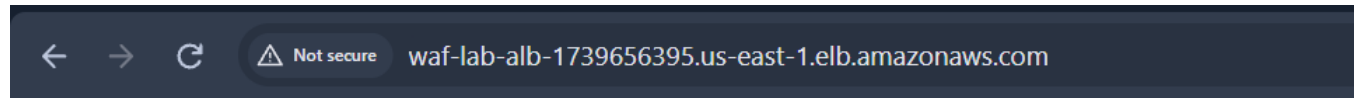
## 9. Troubleshooting

| Issue                    | Cause                                             | Resolution                                                            |
|--------------------------|---------------------------------------------------|-----------------------------------------------------------------------|
| Target group status      | Target initially required ALB association         | Associated the target group with the ALB and verified Healthy status. |
| HTTP 403 during WAF test | Temporary test IP matched the WAF Block rule      | Removed/restored the temporary IP-set test value; access returned.    |
| Duplicate managed rules  | Existing managed rule groups were already present | Inspected and verified existing groups rather than duplicating them.  |

## 10. Final Result

FINAL STATUS — COMPLETED. The AWS WAF Basic Lab was successfully implemented and validated across networking, EC2, ALB, target group, WAF Web ACL, IP-set blocking, managed rules, and monitoring.

## 11. Implementation:

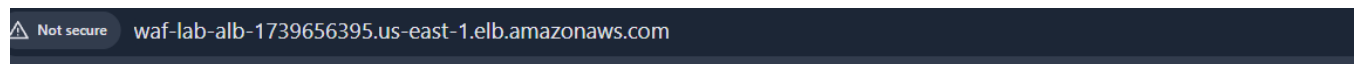


# AWS WAF Lab - Web Server

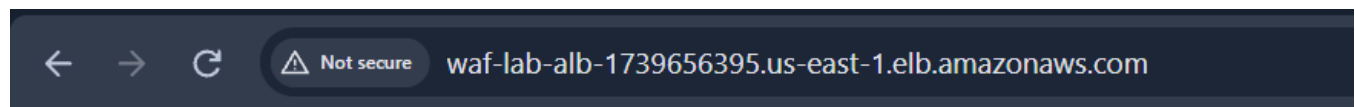
## Application is running successfully

This EC2 instance is protected by AWS WAF through an Application Load Balancer.

Server: waf-lab-web-server



**403 Forbidden**



# AWS WAF Lab - Web Server

## Application is running successfully

This EC2 instance is protected by AWS WAF through an Application Load Balancer.

Server: waf-lab-web-server

waf-lab-second-ip-set

Successfully created IP set waf-lab-second-ip-set

waf-lab-second-ip-set

Delete set

Edit IP set

IP set settings

Name

waf-lab-second-ip-set

Description

IP version

IPv4

Number of addresses

1

Scope

Regional

Region

US East (N. Virginia)

ID

2d8e39fa-c59b-4ed6-9dcb-e5ae850dd320

ARN

arn:aws:wafv2:us-east-1:567487919764:regional/ipset/waf-lab-second-ip-set/2d8e39fa-c59b-4ed6-9dcb-e5ae850dd320

Lock token

694bbe0a-d8ce-401e-84a0-ac2515ee0bd2

IP addresses (1)

Delete

Add address

Filter IP addresses

IP address

192.0.2.1/32

waf-lab-web-acl

Edit protection pack (web ACL)

Add recommended rules based on your traffic insights (3)

AWS Threat Intelligence monitors your previous 2 weeks of allowed traffic for vulnerabilities. Mitigate these by interacting with the specific traffic dimensions below to enable the recommended AWS Managed Rules.

Vulnerable application traffic

0

Allowed Bots traffic

0

Allowed DDoS sources

Rule enabled in default mode

Allowed IP reputation traffic

0

Allowed Anonymous IP traffic

0

waf-lab-web-acl

Filter data

Actions = ALLOW, BLOCK, CAPTCHA, CHALLENGE, MONETIZE

Traffic type = ALL

Clear filters

Last 3 hours

Timezone Local time

Summary

Request counts for all traffic during the specified time range.

Total

8

Allowed

5

Blocked

3

CAPTCHA

0

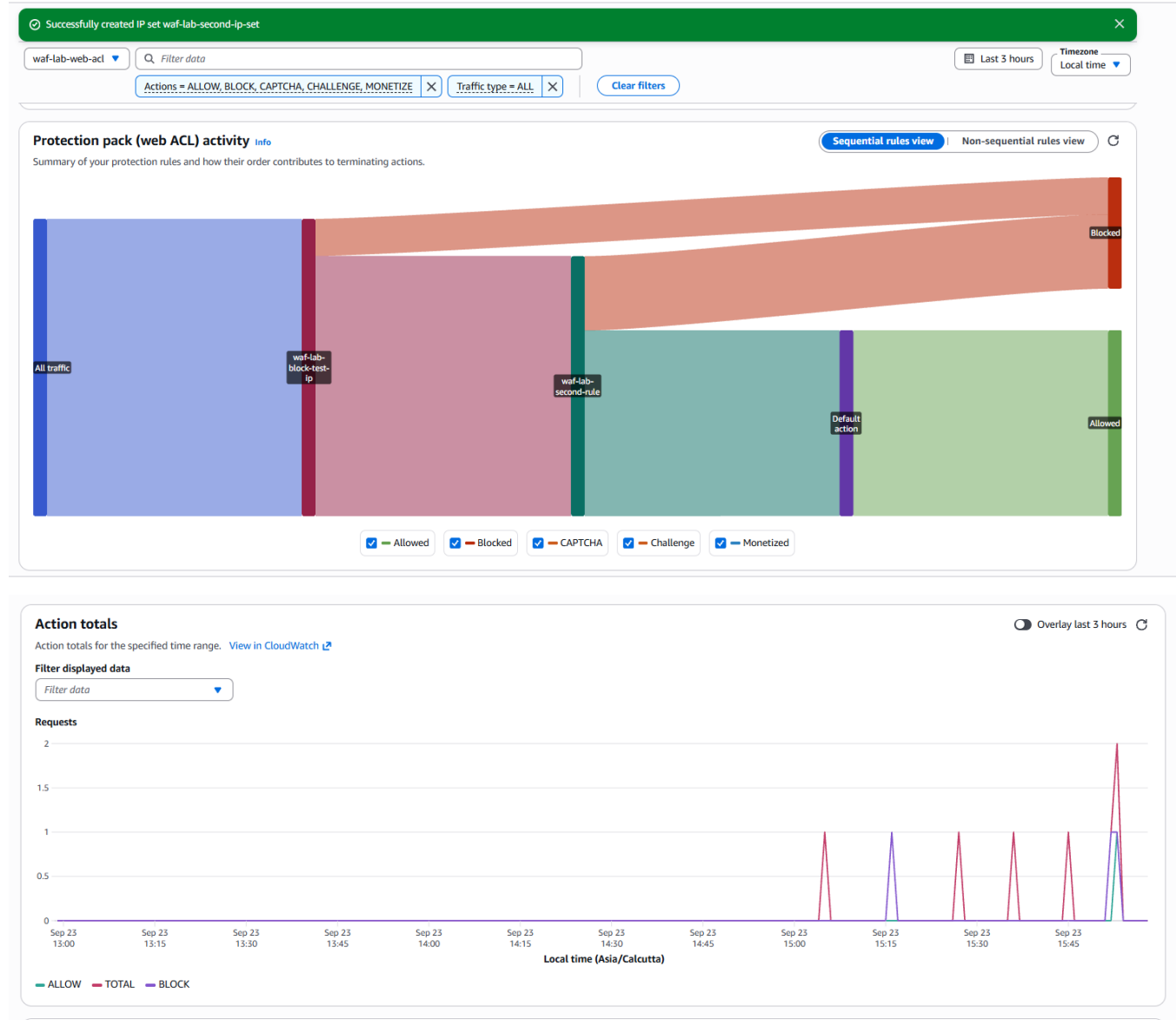
Challenged

0

Monetized

0

nd requests



## 12. Conclusion

The lab provided practical experience implementing AWS WAF with an Application Load Balancer and EC2 web server. The successful IP-blocking test demonstrated that matching requests were intercepted with HTTP 403, while restoring the IP-set configuration returned the application to normal operation.

# Lab 2 — AWS Systems Manager Parameter Store

## Document Overview

---

This document records the completed Parameter Store lab. It preserves the actual parameter names, types, hierarchy, versioning, label, KMS configuration, IAM policy scope, CLI validation, and troubleshooting observed during implementation.

1. Objective
2. Purpose
3. Architecture
4. Prerequisites
5. Implementation
6. Configuration Details
7. Testing & Validation
8. Observations
9. Troubleshooting
10. Final Result
11. Conclusion
12. Completion Checklist

## 1. Objective

Create and validate AWS Systems Manager Parameter Store values using String, StringList, and SecureString types, while demonstrating hierarchical naming, versioning, labels, IAM permissions, KMS configuration, and CLI retrieval.

## 2. Purpose

- Create application configuration parameters under a hierarchy.
- Demonstrate String, StringList, and SecureString parameter types.
- Demonstrate parameter version history and labels.
- Use the AWS-managed Systems Manager KMS key for SecureString.
- Create read-only IAM permissions for the lab hierarchy.
- Validate retrieval from Git Bash using AWS CLI.

### 3. Architecture

IAM User / CLI → IAM Read Policy → AWS Systems Manager Parameter Store (/lab2/app/\*) → String / StringList / SecureString

| Component           | Implemented Configuration         |
|---------------------|-----------------------------------|
| Region              | us-east-1 — US East (N. Virginia) |
| Parameter hierarchy | /lab2/app/*                       |
| String              | /lab2/app/name                    |
| StringList          | /lab2/app/allowed-ports           |
| SecureString        | /lab2/app/db-password             |
| KMS key             | alias/aws/ssm                     |
| Read policy         | Lab2-ParameterStore-ReadOnly      |

### 4. Prerequisites

- AWS account and console access
- AWS Region us-east-1
- Permission to create Parameter Store parameters, IAM policies, and perform CLI validation
- AWS CLI configured for the lab IAM identity

### 5. Implementation

#### 5.1 Create String Parameter

Created /lab2/app/name as a Standard String parameter with initial value AWS-ParameterStore-Lab. The parameter was later edited to AWS-ParameterStore-Lab-v2, creating Version 2.

#### 5.2 Create StringList Parameter

Created /lab2/app/allowed-ports as a Standard StringList with value 80,443,8080.

#### 5.3 Create SecureString Parameter

Created /lab2/app/db-password as a Standard SecureString. KMS source was the current account and the AWS-managed Systems Manager key alias/aws/ssm was selected. The demo secret value is intentionally not reproduced.

#### 5.4 Verify Hierarchy

The parameter names /lab2/app/name, /lab2/app/allowed-ports, and /lab2/app/db-password established the /lab2/app hierarchy.

#### 5.5 Versioning and Label

The /lab2/app/name parameter was edited, producing Version 2. Parameter history confirmed Versions 1 and 2. Version 2 was labeled current.

#### 5.6 Create IAM Read Policy

Created Lab2-ParameterStore-ReadOnly with read-only SSM actions. The final policy included both the parent hierarchy resource and child resources so GetParametersByPath could be evaluated correctly.

## 5.7 CLI Validation

Validated String, StringList, SecureString, and GetParametersByPath retrieval from Git Bash. The SecureString plaintext was not reproduced.

## 6. Configuration Details

| Parameter               | Type         | Tier     | Configuration                                         |
|-------------------------|--------------|----------|-------------------------------------------------------|
| /lab2/app/name          | String       | Standard | AWS-ParameterStore-Lab-v2 / Version 2 / label current |
| /lab2/app/allowed-ports | StringList   | Standard | 80,443,8080                                           |
| /lab2/app/db-password   | SecureString | Standard | Demo secret; plaintext intentionally omitted          |

### 6.1 IAM Policy

| Setting         | Final Value                                                   |
|-----------------|---------------------------------------------------------------|
| Policy name     | Lab2-ParameterStore-ReadOnly                                  |
| Policy ARN      | arn:aws:iam::567487919764:policy/Lab2-ParameterStore-ReadOnly |
| Actions         | ssm:GetParameter; ssm:GetParameters; ssm:GetParametersByPath  |
| Parent resource | arn:aws:ssm:us-east-1:567487919764:parameter/lab2/app         |
| Child resources | arn:aws:ssm:us-east-1:567487919764:parameter/lab2/app/*       |

### 6.2 KMS Configuration

The SecureString parameter used the current AWS account as the KMS source and the AWS-managed Systems Manager key identified as alias/aws/ssm.

## 7. Testing & Validation

| Test                   | Validation                                                               | Result |
|------------------------|--------------------------------------------------------------------------|--------|
| String retrieval       | /lab2/app/name returned AWS-ParameterStore-Lab-v2                        | PASS   |
| StringList retrieval   | /lab2/app/allowed-ports returned 80,443,8080                             | PASS   |
| SecureString retrieval | Retrieved with decryption during validation; plaintext not documented    | PASS   |
| Version history        | Versions 1 and 2 confirmed                                               | PASS   |
| Current label          | Version 2 labeled current                                                | PASS   |
| GetParametersByPath    | /lab2/app hierarchy successfully retrieved after policy scope correction | PASS   |

## 8. Observations

- Parameter Store hierarchy organizes related application settings under /lab2/app.
- StringList stores multiple comma-separated values in a single parameter.
- SecureString provides encrypted parameter storage using KMS.

## SUNITH – AWS SELF-LEARNING AND HANDS-ON IMPLEMENTATION ACTIVITY

- Parameter edits create new versions rather than replacing history.
- Labels can identify a selected parameter version.
- GetParametersByPath required the IAM policy to include the parent hierarchy resource.

## 9. Troubleshooting

| Issue                            | Cause                                                                       | Resolution                                                                |
|----------------------------------|-----------------------------------------------------------------------------|---------------------------------------------------------------------------|
| Git Bash SSM AccessDenied        | MSYS path conversion changed /lab2/app/name into a Windows filesystem path. | Used MSYS_NO_PATHCONV=1 for affected local AWS CLI commands.              |
| GetParametersByPath AccessDenied | Policy covered child parameters but not the parent /lab2/app resource.      | Added both parameter/lab2/app and parameter/lab2/app/* to the IAM policy. |

## 10. Final Result

FINAL STATUS — COMPLETED. Parameter Store parameters, hierarchy, SecureString/KMS configuration, versioning, label, IAM read permissions, and CLI validation were successfully completed.

# 11. Implementation:

## Create parameter

Parameter details

Name

Q

/lab2/app/name

X

When naming a parameter, you can use forward slashes (/) to organize it into a hierarchy. [Learn more about hierarchies](#)

Description — Optional

Application name for Parameter Store lab

Tier

Parameter Store offers standard and advanced parameters.

Tip: Set your preferred default tier

Enable Advanced Tier to support larger parameter sizes and unlock additional features.

Manage settings

Standard

Use for: Configuration values

- Store up to 10,000 parameters
- Parameter size up to 4 KB
- No additional charge

Advanced

Premium

Use for: Production workloads, large configurations, compliance

- Cross-account sharing
- Store up to 100,000 parameters
- Parameter size up to 8 KB (2x storage)
- Support expiration and "no change" notifications

Type

String

Any string value.

StringList

Separate strings using commas.

SecureString

Encrypt sensitive data using KMS keys from your account or another account.

Data type

text

Value

## Create parameter

Parameter details

Name

Q

/lab2/app/allowed-ports

X

When naming a parameter, you can use forward slashes (/) to organize it into a hierarchy. [Learn more about hierarchies](#)

Description — Optional

Allowed application ports for Parameter Store lab

Tier

Parameter Store offers standard and advanced parameters.

Tip: Set your preferred default tier

Enable Advanced Tier to support larger parameter sizes and unlock additional features.

Manage settings

Standard

Use for: Configuration values

- Store up to 10,000 parameters
- Parameter size up to 4 KB
- No additional charge

Advanced

Premium

Use for: Production workloads, large configurations, compliance

- Cross-account sharing
- Store up to 100,000 parameters
- Parameter size up to 8 KB (2x storage)
- Support expiration and "no change" notifications

Type

String

Any string value.

StringList

Separate strings using commas.

SecureString

Encrypt sensitive data using KMS keys from your account or another account.

Value

80,443,8080

© 2020 Amazon.com, Inc. or its affiliates. All rights reserved. AWS and the AWS logo are trademarks of Amazon.com, Inc. or its affiliates. All other marks are the property of their respective owners.

```

Samal@Sneha@DESKTOP-KOF17A5 MINGW64 ~/Downloads
$ MSYS_NO_PATHCONV=1 aws ssm get-parameter \
  --name "/lab2/app/name" \
  --region us-east-1
{
  "Parameter": {
    "Name": "/lab2/app/name",
    "Type": "String",
    "Value": "AWS-ParameterStore-Lab-v2",
    "Version": 2,
    "LastModifiedDate": "2026-09-23T16:52:38.485000+05:30",
    "ARN": "arn:aws:ssm:us-east-1:567487919764:parameter/lab2/app/name",
    "DataType": "text"
  }
}

Samal@Sneha@DESKTOP-KOF17A5 MINGW64 ~/Downloads
$ |

```

```

Samal@Sneha@DESKTOP-KOF17A5 MINGW64 ~/Downloads
$ MSYS_NO_PATHCONV=1 aws ssm get-parameter \
  --name "/lab2/app/allowed-ports" \
  --region us-east-1
{
  "Parameter": {
    "Name": "/lab2/app/allowed-ports",
    "Type": "StringList",
    "Value": "80,443,8080",
    "Version": 1,
    "LastModifiedDate": "2026-09-23T16:48:00.874000+05:30",
    "ARN": "arn:aws:ssm:us-east-1:567487919764:parameter/lab2/app/allowed-ports",
    "DataType": "text"
  }
}

Samal@Sneha@DESKTOP-KOF17A5 MINGW64 ~/Downloads
$ |

```

## 12. Conclusion

This lab demonstrated how AWS Systems Manager Parameter Store can centralize application configuration and protected values. The completed work covered multiple parameter types, hierarchical naming, version history, labels, KMS-backed SecureString storage, IAM read permissions, and command-line validation.

# Lab 3 — EC2 + Parameter Store Integration

## Document Overview

---

This document records the completed Lab 3 implementation: integrating the existing Amazon EC2 web server with AWS Systems Manager Parameter Store through an EC2 IAM role. The work reused the Lab 1 application infrastructure and validated retrieval of String, StringList, and SecureString parameters, including hierarchical retrieval with GetParametersByPath.

The documentation follows the same practical structure used for the Lab 1 technical record and contains only the configuration and validation actually performed. Sensitive credentials and the SecureString plaintext value are intentionally not reproduced.

1. Objective
2. Purpose
3. Architecture
4. Prerequisites
5. Implementation
6. Configuration Details
7. Testing & Validation
8. Observations
9. Troubleshooting
10. Final Result
11. Conclusion
12. Completion Checklist

### 1. Objective

The objective of this lab is to integrate the existing Amazon EC2 web server with AWS Systems Manager Parameter Store using an IAM instance role. The EC2 instance must be able to retrieve application configuration and a SecureString parameter without storing long-lived AWS access keys on the server.

### 2. Purpose

- Attach an IAM role to an existing EC2 instance.
- Reuse the Lab 1 EC2/ALB/WAF environment instead of creating new application infrastructure.
- Retrieve String and StringList configuration values from Parameter Store.
- Validate access to a SecureString parameter with decryption enabled without exposing its plaintext value.
- Validate GetParametersByPath against the /lab2/app hierarchy.

- Record and resolve IAM permission and CLI-related issues encountered during implementation.

### 3. Architecture

Internet → Application Load Balancer → Target Group → Existing EC2 / Apache → EC2 IAM Instance Role → AWS Systems Manager Parameter Store (/lab2/app/\*)

| Layer               | Implemented Configuration              |
|---------------------|----------------------------------------|
| VPC                 | waf-lab-vpc / 10.0.0.0/16              |
| EC2                 | waf-lab-web-server / Amazon Linux 2023 |
| ALB                 | waf-lab-alb / Internet-facing / IPv4   |
| Target Group        | waf-lab-tg / HTTP :80 / health check / |
| WAF                 | waf-lab-web-acl / Regional             |
| IAM Role            | Lab3-EC2-ParameterStore-Role           |
| Permission Policy   | Lab2-ParameterStore-ReadOnly           |
| Parameter Hierarchy | /lab2/app/*                            |

### 4. Prerequisites

- AWS account and console access
- AWS Region: us-east-1
- Existing Lab 1 EC2 instance: waf-lab-web-server
- Existing Parameter Store parameters under /lab2/app/
- Permission to create an IAM role and attach it to the EC2 instance
- EC2 Instance Connect or another terminal method for command validation

### 5. Implementation

#### 5.1 Create the EC2 IAM Role

Created an IAM role specifically for EC2 and configured EC2 as the trusted service. The existing read-only Parameter Store policy from Lab 2 was attached to the role.

| Setting           | Value                        |
|-------------------|------------------------------|
| Role              | Lab3-EC2-ParameterStore-Role |
| Trusted service   | EC2                          |
| Permission policy | Lab2-ParameterStore-ReadOnly |

#### 5.2 Attach the Role to the Existing EC2

The role Lab3-EC2-ParameterStore-Role was attached to the existing instance waf-lab-web-server. No additional EC2 instance was launched. The existing ALB, target group, security groups, and WAF configuration were left unchanged.

#### 5.3 Validate the IAM Identity from EC2

From the Amazon Linux EC2 terminal, the following command was executed:

```
aws sts get-caller-identity
```

The returned identity was an assumed-role identity for Lab3-EC2-ParameterStore-Role, confirming that the instance was using the attached IAM role.

## 5.4 Retrieve the String Parameter

Command used:

```
aws ssm get-parameter \  
--name "/lab2/app/name" \  
--region us-east-1
```

Validation result: the command succeeded and returned the Version 2 value AWS-ParameterStore-Lab-v2.

## 5.5 Retrieve the StringList Parameter

Command used:

```
aws ssm get-parameter \  
--name "/lab2/app/allowed-ports" \  
--region us-east-1
```

Validation result: the command succeeded and returned 80,443,8080.

## 5.6 Validate SecureString Access

To validate SecureString access while avoiding exposure of the secret, only the parameter type was queried:

```
aws ssm get-parameter \  
--name "/lab2/app/db-password" \  
--with-decryption \  
--region us-east-1 \  
--query "Parameter.Type"
```

Validation result: "SecureString". The plaintext secret was intentionally not reproduced.

## 5.7 Validate Hierarchical Retrieval

The initial GetParametersByPath request returned AccessDenied. The existing IAM policy covered the child parameter resources but did not include the parent path resource required for this operation.

The policy was updated to include both:

arn:aws:ssm:us-east-1:567487919764:parameter/lab2/app

arn:aws:ssm:us-east-1:567487919764:parameter/lab2/app/\*

After the policy update, the following command succeeded:

```
aws ssm get-parameters-by-path \  
--path "/lab2/app" \  
--region us-east-1 \  
--query "Parameters[].{Name:Name,Type:Type}"
```

# 6. Configuration Details

| Component        | Final Configuration          |
|------------------|------------------------------|
| EC2 Instance     | waf-lab-web-server           |
| Operating System | Amazon Linux 2023            |
| IAM Role         | Lab3-EC2-ParameterStore-Role |

|                           |                               |
|---------------------------|-------------------------------|
| Trusted Service           | EC2                           |
| Permission Policy         | Lab2-ParameterStore-ReadOnly  |
| Parameter Path            | /lab2/app                     |
| String Parameter          | /lab2/app/name                |
| StringList Parameter      | /lab2/app/allowed-ports       |
| SecureString Parameter    | /lab2/app/db-password         |
| GetParametersByPath Scope | Parent path + child resources |

## 6.1 Final IAM Policy Scope

The final Lab2-ParameterStore-ReadOnly policy allowed the following read operations:

- ssm:GetParameter
- ssm:GetParameters
- ssm:GetParametersByPath

Final resource scope:

arn:aws:ssm:us-east-1:567487919764:parameter/lab2/app

arn:aws:ssm:us-east-1:567487919764:parameter/lab2/app/\*

## 7. Testing & Validation

| Test                 | Validation                                                          | Result |
|----------------------|---------------------------------------------------------------------|--------|
| IAM identity         | EC2 returned assumed-role identity for Lab3-EC2-ParameterStore-Role | PASS   |
| String retrieval     | /lab2/app/name returned AWS-ParameterStore-Lab-v2                   | PASS   |
| StringList retrieval | /lab2/app/allowed-ports returned 80,443,8080                        | PASS   |
| SecureString access  | Parameter Type returned SecureString with --with-decryption         | PASS   |
| GetParametersByPath  | /lab2/app hierarchy retrieval succeeded after policy update         | PASS   |
| Existing application | Lab 1 EC2/ALB/WAF environment reused without new EC2                | PASS   |

## 8. Observations

- An EC2 instance can obtain temporary AWS credentials through its attached IAM role rather than using long-lived access keys.
- The Lab 2 read policy could be reused by the EC2 role for Parameter Store access.
- String and StringList parameters were retrieved directly from the EC2 terminal.
- SecureString access was validated without recording the plaintext secret.
- GetParametersByPath required the IAM policy to include the parent hierarchy resource as well as child parameter resources.
- The existing ALB, target group, security groups, and WAF configuration did not need to be changed for the Parameter Store integration.

## 9. Troubleshooting

| Issue                                                   | Cause                                                                       | Resolution                                                                |
|---------------------------------------------------------|-----------------------------------------------------------------------------|---------------------------------------------------------------------------|
| GetParametersByPath<br>AccessDenied from EC2            | Policy covered parameter children but not the parent /lab2/app resource.    | Added both parameter/lab2/app and parameter/lab2/app/* to the IAM policy. |
| Local Git Bash SSM AccessDenied during Lab 2 validation | MSYS path conversion changed /lab2/app/name into a Windows filesystem path. | Used MSYS_NO_PATHCONV=1 for the affected local Git Bash commands.         |

The Git Bash path-conversion issue belonged to the Lab 2 CLI validation and is recorded here because it directly affected the Parameter Store permissions workflow used by Lab 3.

## 10. Final Result

### FINAL STATUS — COMPLETED

The EC2 + Parameter Store Integration Lab was successfully implemented and validated. The existing waf-lab-web-server instance uses Lab3-EC2-ParameterStore-Role to retrieve Parameter Store values under /lab2/app. String, StringList, SecureString access, and GetParametersByPath were successfully validated.

Final flow: Application Load Balancer → Target Group → Existing EC2 → IAM Instance Role → AWS Systems Manager Parameter Store

## 10. Implementation:

```
[ec2-user@ip-10-0-1-27 ~]$ aws ssm get-parameter --name "/lab2/app/name" --region us-east-1
{
  "Parameter": {
    "Name": "/lab2/app/name",
    "Type": "String",
    "Value": "AWS-ParameterStore-Lab-v2",
    "Version": 2,
    "LastModifiedDate": "2026-09-23T11:22:38.485000+00:00",
    "ARN": "arn:aws:ssm:us-east-1:567487919764:parameter/lab2/app/name",
    "DataType": "text"
  }
}
[ec2-user@ip-10-0-1-27 ~]$ aws ssm get-parameter --name "/lab2/app/allowed-ports" --region us-east-1
{
  "Parameter": {
    "Name": "/lab2/app/allowed-ports",
    "Type": "StringList",
    "Value": "80,443,8080",
    "Version": 1,
    "LastModifiedDate": "2026-09-23T11:18:00.874000+00:00",
    "ARN": "arn:aws:ssm:us-east-1:567487919764:parameter/lab2/app/allowed-ports",
    "DataType": "text"
  }
}
[ec2-user@ip-10-0-1-27 ~]$ aws ssm get-parameter --name "/lab2/app/db-password" --with-decryption --region us-east-1 --query "Parameter.Type"
"SecureString"
[ec2-user@ip-10-0-1-27 ~]$

[ec2-user@ip-10-0-1-27 ~]$ aws ssm get-parameters-by-path --path "/lab2/app" --region us-east-1 --query "Parameters[0].{Name:Name,Type:Type}"
[
  {
    "Name": "/lab2/app/allowed-ports",
    "Type": "StringList"
  },
  {
    "Name": "/lab2/app/db-password",
    "Type": "SecureString"
  },
  {
    "Name": "/lab2/app/name",
    "Type": "String"
  }
]
[ec2-user@ip-10-0-1-27 ~]$
```

## 12. Conclusion

This lab demonstrated a practical AWS integration pattern in which an existing EC2 application retrieves centralized configuration and secret data from Systems Manager Parameter Store through an IAM instance role. The implementation reused the completed Lab 1 infrastructure and the Parameter Store resources created in Lab 2.

The successful validation confirmed role-based identity from EC2, read access to String and StringList parameters, SecureString access with decryption, and hierarchical parameter retrieval. The permission issue encountered with `GetParametersByPath` was resolved by extending the resource scope to include both the hierarchy parent and its child parameters.